# 06 — Deployment Infrastructure (Remote Backend)

**Revision:** 1 **Parent:** `00_MASTER_INDEX.md` **Prerequisites:** A-07 through A-11 (operator deployment decisions) must be resolved. A-09 (lets_encrypt + sftp) must be resolved.

---

## Overview

Deploy the full Helix OTA stack to the production remote host, with TLS, monitoring, backups, and artifact publishing. The scaffolding exists at `scripts/remote_deploy/` + `deploy/svord/`. It has been locally validated (parse gates, compose config, container build, redacted dry-runs) but NEVER run against a live server.

**Architecture:**

```
Operator → SSH → Remote host (hxota)
  ├── podman-compose (rootless)
  │   ├── postgres:16-alpine
  │   ├── minio (S3-compatible object storage)
  │   ├── ota-server (Go/Gin control plane)
  │   └── nginx proxy (TLS termination, SPA serving)
  ├── Prometheus + Grafana (monitoring)
  ├── Promtail → Loki (log aggregation)
  └── systemd user units (startup, health checks)
```

---

## F-01 [OPERATOR] — Resolve Deployment Decisions

Already documented in `01_OPERATOR_DECISIONS.md` §7–§11. Must be resolved before any deployment work.

---

# F-02 [OPERATOR/AGENT] — Bootstrap Remote Host

**Effort:** M (~2h, mostly one-time operator setup)

## One-time host preparation (operator):

```
# Create deploy user
sudo useradd -m hxota
sudo passwd hxota

# Install rootless podman + podman-compose
sudo apt install podman podman-compose  # Ubuntu 24.04

# Enable lingering (user services survive logout)
sudo loginctl enable-linger hxota

# Allow unprivileged port binding (for ports 80/443)
sudo sysctl net.ipv4.ip_unprivileged_port_start=80
echo "net.ipv4.ip_unprivileged_port_start=80" | sudo tee /etc/
        sysctl.d/99-hxota.conf

# Set up firewall (allow SSH, HTTPS, HTTP)
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw allow 80/tcp
sudo ufw enable
```

## Deploy directory structure (agent, via deploy scripts):

```
# The deploy scripts create this structure on the remote:
~/hxota-stack/
├── compose.svord.yml
├── stack.env          (chmod 600)
├── nginx/
│   └── hxota-proxy.conf
├── certs/
│   └── <domain>/
│       ├── fullchain.pem
│       └── privkey.pem
├── srv/
│   ├── website/       (Angular SSR build)
│   ├── console/       (ota-manager build)
```

```
│   ├── dashboard/      (dashboard build)
│   └── acme/           (ACME challenge files)
└── server/
    └── Dockerfile      (build context)
```

---

# F-03 [AGENT] — Incorporate lets_encrypt + sftp Submodules

**Effort:** S (~30 min) **Source:** `docs/remote_deploy/REMOTE_DEPLOY.md` §5

## What to do:

1. **Add lets_encrypt submodule:**

   ```
   git submodule add git@github.com:vasic-digital/lets_encrypt.git
           submodules/lets_encrypt
   cd submodules/lets_encrypt && bash upstreams/
           install_upstreams.sh && cd ../..
   ```

2. **Add sftp submodule:**

   ```
   git submodule add git@github.com:vasic-digital/sftp.git
           submodules/sftp
   cd submodules/sftp && bash upstreams/install_upstreams.sh &&
           cd ../..
   ```

3. **Update helix-deps.yaml** with entries for both.

4. **Confirm CLI entrypoints** (§11.4.99): Read each submodule's README, confirm the actual CLI command. Update `https_certs.sh` and `publish_artifacts.sh` to use the real entrypoints.

5. **Update `.gitmodules`** — already done by `git submodule add`.

---

# F-04 [AGENT/OPERATOR] — Provision TLS Certificates

**Effort:** M (~1h) **Source:** `docs/remote_deploy/REMOTE_DEPLOY.md` §4

## What to do:

1. **Configure domains:** `hxota.dev` (console + API), `hxota.com` (website).

2. **Issue certs via lets_encrypt:**

```
bash scripts/remote_deploy/https_certs.sh issue --domain
        hxota.dev --email admin@hxota.com
bash scripts/remote_deploy/https_certs.sh issue --domain
        hxota.com --email admin@hxota.com
```

3. **Verify certs:** `openssl x509 -in certs/hxota.dev/fullchain.pem -text -noout`

4. **Set up auto-renewal:** Cron job running `https_certs.sh renew` weekly.

---

# F-05 [AGENT] — Deploy Full Stack to Remote Host

**Effort:** M (~2h) **Source:** `docs/remote_deploy/REMOTE_DEPLOY.md`, `scripts/remote_deploy/deploy.sh`

## What to do:

1. **Populate deploy env:** Copy `scripts/remote_deploy/.hxota_deploy.env.example` to `scripts/testing/secrets/.hxota_deploy.env`. Fill in ALL values (see A-08, A-11).

2. **Run the orchestrator:**

```
bash scripts/remote_deploy/deploy.sh
```

   This runs the full 6-stage pipeline:

   - Preflight (SSH check, remote dir creation)
   - Infrastructure (postgres + minio, stack.env)
   - API (cross-compile ota-server, rsync, build, bring-up)
   - Dashboards (build SPAs, stage into srv/)
   - Website (build Angular app, stage into srv/)
   - Certs (issue/renew TLS certs)
   - Bring-up (podman-compose up -d)
   - Health confirm (live 200 check on /healthz + /readyz)

3. **Verify stack is healthy:**

```
bash scripts/remote_deploy/status.sh
curl https://hxota.dev/healthz
curl https://hxota.dev/readyz
```

4. **If issues:** Check `podman logs ota-server`, `podman logs postgres`, `podman logs proxy`.

---

# F-06 [AGENT] — Configure PostgreSQL Backups

**Effort:** S (~30 min) **Source:** `server/deploy/backup.sh`, Gap tracker G-57

## What to do:

1. **Review backup script:** `server/deploy/backup.sh` — does `pg_dump -Fc` and uploads to S3/MinIO.
2. **Configure S3 credentials** for backup destination (MinIO bucket `helix-backups` or similar).
3. **Set up cron job** on the remote host (as `hxota` user):

   ```
   0 2 * * * /home/hxota/hxota-stack/backup.sh >> /home/hxota/hxota-stack/backup.log 2>&1
   ```

4. **Test backup + restore:** Run backup, then test restore to a clean database per `server/deploy/restore.md`.
5. **Verify:** Backup file exists, can be restored, data is consistent.

---

# F-07 [AGENT] — Wire Monitoring Stack (Prometheus + Grafana + Loki)

**Effort:** M (~2h) **Source:** Gap tracker G-60, G-61

## Current state:

Prometheus alert rules exist (`server/deploy/prometheus/alerts.yml`), Grafana dashboard JSON exists (`server/deploy/grafana/ota-dashboard.json`), Promtail config exists (`server/deploy/promtail.yml`). These are config FILES — they need to be deployed as running services.

### What to do:

1. **Add monitoring services to compose:**

```yaml
# In deploy/svord/compose.svord.yml
prometheus:
  image: prom/prometheus:latest
  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/
        prometheus.yml
    - ./prometheus/alerts.yml:/etc/prometheus/alerts.yml
    - prometheus_data:/prometheus

grafana:
  image: grafana/grafana:latest
  volumes:
    - ./grafana/dashboards:/etc/grafana/provisioning/dashboards
    - grafana_data:/var/lib/grafana

loki:
  image: grafana/loki:latest

promtail:
  image: grafana/promtail:latest
  volumes:
    - ./promtail.yml:/etc/promtail/promtail.yml
    - /var/log:/var/log:ro  # or podman logs
```

2. **Configure Prometheus:** Scrape `ota-server:8080/metrics`, postgres exporter.
3. **Configure Grafana:** Import the OTA dashboard JSON, set up data sources.
4. **Configure Promtail:** Ship logs to Loki.
5. **Verify:** Alert rules fire correctly (test by taking down PostgreSQL). Dashboard shows metrics.

---

# F-08 [AGENT] — Deploy Dashboard, Console, Website SPAs

**Effort:** S (~30 min)

### What to do:

1. **Dashboard:** Build and deploy to remote host:

   ```
   cd dashboard && pnpm build
   rsync -avz dist/ hxota@<host>:~/hxota-stack/srv/dashboard/
   ```

2. **Console (ota-manager):** Build and deploy:

   ```
   cd clients/ota-manager && pnpm build
   rsync -avz dist/ hxota@<host>:~/hxota-stack/srv/console/
   ```

3. **Website:** Build Angular SSR and deploy:

   ```
   bash scripts/remote_deploy/deploy_website.sh
   ```

4. **Reload proxy:** `podman exec proxy nginx -s reload`

5. **Verify:** Browser access to `https://hxota.dev/`, `https://hxota.com/`, `https://hxota.dev/dashboard/`.

---

## F-09 [AGENT/OPERATOR] — SSH Key Auth + Credential Rotation + Firewall

**Effort:** M (~1h)

### What to do:

1. **Switch to SSH key auth** (if decided in A-08b):

   ```
   ssh-keygen -t ed25519 -f ~/.ssh/hxota_prod -C "hxota-prod-
           deploy"
   ssh-copy-id -i ~/.ssh/hxota_prod.pub hxota@<host>
   ```

2. **Update deploy env:** Set `HXOTA_SSH_USE_PASSWORD=0`, `HXOTA_SSH_KEY=~/.ssh/hxota_prod`.

3. **Rotate all credentials:**

   - PostgreSQL password: `openssl rand -base64 48`
   - MinIO access/secret keys: `openssl rand -base64 48` each
   - Token secret: `openssl rand -base64 48`
   - Update `stack.env` on remote host, restart stack.

4. **Lock down SSH:** Disable password auth, disable root login:

```
sudo sed -i 's/^#PasswordAuthentication yes/
        PasswordAuthentication no/' /etc/ssh/sshd_config
sudo sed -i 's/^#PermitRootLogin prohibit-password/
        PermitRootLogin no/' /etc/ssh/sshd_config
sudo systemctl restart sshd
```

5. **Verify firewall:** Only 22, 80, 443 open. `sudo ufw status verbose`.

---

# F-10 [AGENT] — Production Smoke Test

**Effort:** M (~1h)

## What to do:

1. **Health probes:**

```
curl -s https://hxota.dev/healthz        # → {"status":"ok"}
curl -s https://hxota.dev/readyz          # →
        {"status":"ready"}
curl -s https://hxota.dev/api/v1/metrics  # → Prometheus
        metrics
```

2. **Auth:**

```
curl -s -X POST https://hxota.dev/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin@helix.example","password":"<from-
        deploy-env>"}'
# → {"access_token":"...", "refresh_token":"..."}
```

3. **Artifact upload:**

```
# Generate test artifact
go run scripts/testing/gen_key.go
go run scripts/testing/sign_artifact.go --key private.key --
        input test.bin --output test.zip


# Upload
curl -s -X POST https://hxota.dev/api/v1/artifacts/upload \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@test.zip" \
  -F "metadata={\"os_type\":\"android\",\"target_model\":\"rk3588_t\",\"version
# → 201 {"artifact_id":"..."}
```

4. **Device registration:**

```
curl -s -X POST https://hxota.dev/api/v1/devices/register \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"hardware_id":"test-device-001","model":"Orange Pi 5
        Max","os_type":"android"}'
# → 201 {"device_id":"...", "token":"..."}
```

5. **OTA cycle:** Create release → create deployment → device update check → 200 with update offer.
6. **All 5 smoke tests must PASS** before declaring the deployment live.

---

# Verification Checklist

| Step | Action | Expected Result |
|---|---|---|
| F-02 | Host bootstrapped | hxota user exists, podman runs rootless, lingering enabled |
| F-03 | Submodules added | lets_encrypt and sftp directories exist, CLI confirmed |
| F-04 | TLS certs issued | HTTPS works on both domains, cert valid 90 days |
| F-05 | Stack deployed | All 4 services running, /healthz 200, /readyz 200 |
| F-06 | Backups configured | Cron job exists, restore test passes |
| F-07 | Monitoring wired | Prometheus scraping, Grafana dashboards visible, Loki ingesting |
| F-08 | SPAs deployed | Dashboard, console, website all reachable over HTTPS |
| F-09 | Security hardened | SSH key-only, no password auth, firewall locked down |
| F-10 | Smoke test | All 5 tests PASS (health, auth, upload, register, OTA cycle) |

---

# Danger Zones

| # | Danger | Mitigation |
|---|---|---|
| DZ-F1 | | Verify B-01 first — never deploy with default token secret |

| # | Danger | Mitigation |
|---|---|---|
| | Deploying before SEC-1 fail-fast (B-01) is verified | |
| DZ-F2 | Deploying without TLS → credentials travel in plaintext | TLS certs (F-04) are MANDATORY before any traffic |
| DZ-F3 | Rootful podman/ docker → violates §11.4.161 | Use ONLY rootless podman; `podman info \| grep rootless` must show true |
| DZ-F4 | Hardcoded IPs/ hostnames in scripts → breaks on host change | All addresses from deploy env (`$ADDRESS_HXOTA`), never hardcoded |
| DZ-F5 | Backup not tested → false sense of security | Run a FULL restore test before going live |

# Honest Boundary (§11.4.6)

- The deploy scaffolding (`scripts/remote_deploy/`) has been validated locally (parse + compose config + container build + dry-run). Zero live deploys have been performed.
- lets_encrypt and sftp submodules are NOT on disk (verified this session). Their CLIs are UNCONFIRMED (§11.4.99) — must be confirmed against README before use.
- The monitoring stack configs (Prometheus/Grafana/Loki) exist as files but have NOT been deployed as running services.
- The ota-server Docker image build used a STUB binary in local validation. The real build requires cross-compilation via `deploy_api.sh`.